Introduction to pin muxing



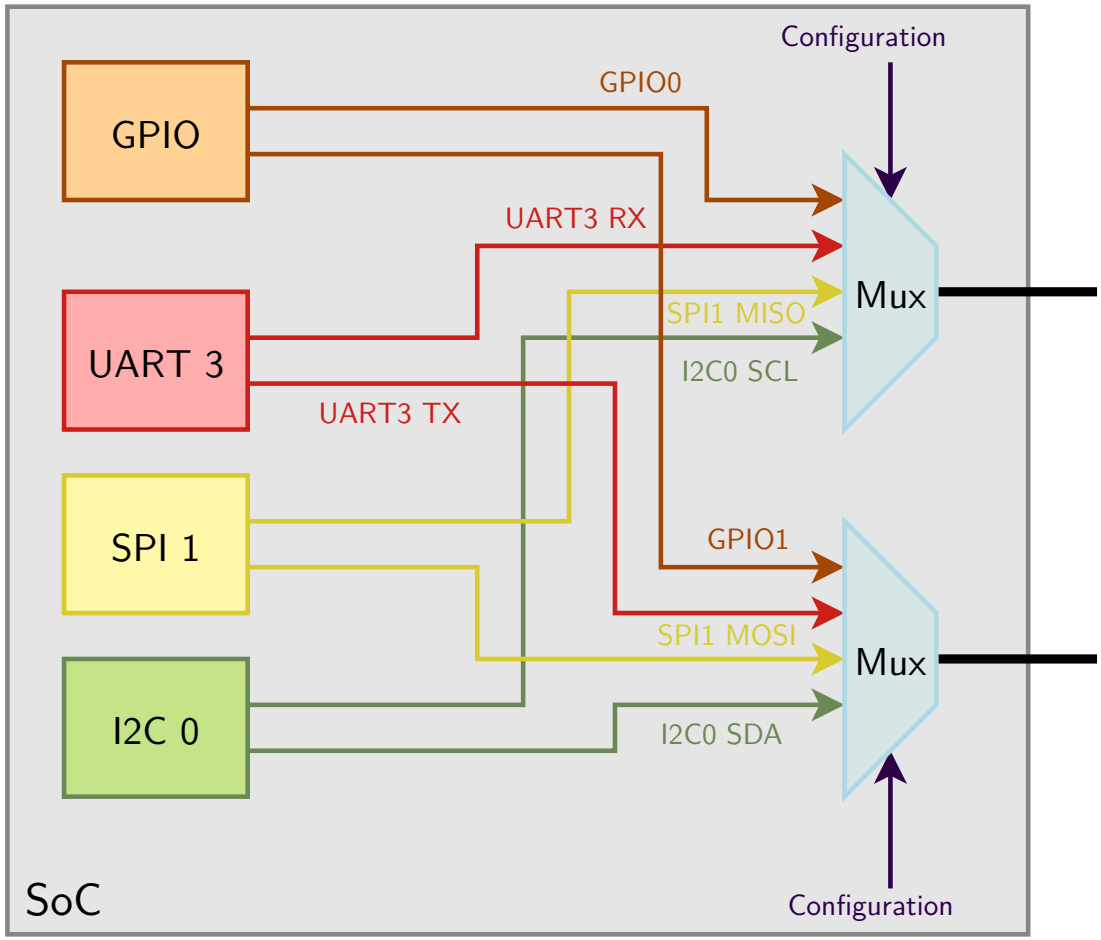
© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



What is pin muxing?

- Modern SoCs (System on Chip) include more and more hardware blocks, many of which need to interface with the outside world using *pins*.
- However, the physical size of the chips remains small, and therefore the number of available pins is limited.
- For this reason, not all of the internal hardware block features can be exposed on the pins simultaneously.
- The pins are **multiplexed**: they expose either the functionality of hardware block A **or** the functionality of hardware block B.
- This *multiplexing* is usually software configurable.

Pin muxing diagram



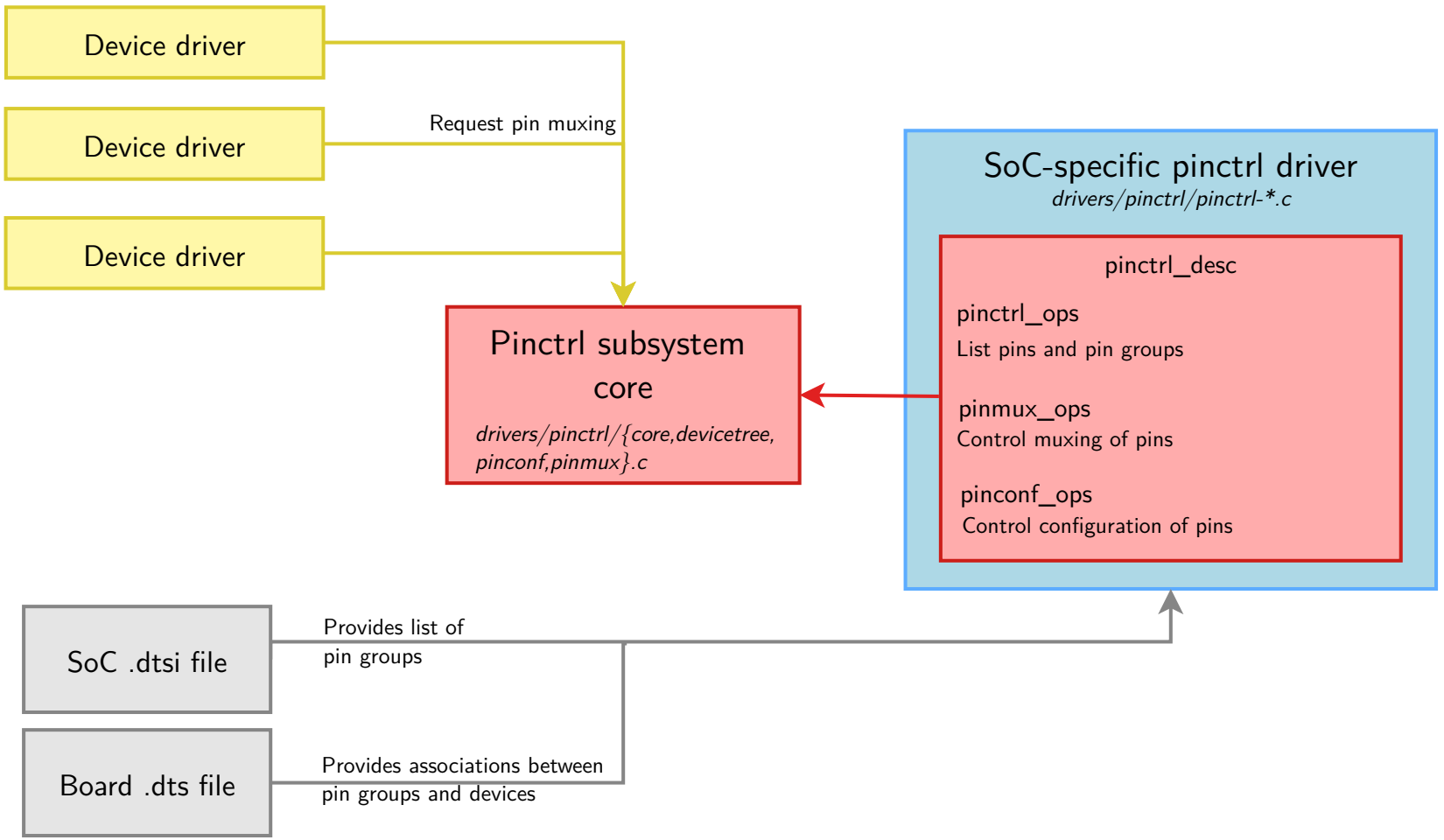


Pin muxing in the Linux kernel

- Since Linux 3.2, a `pinctrl` subsystem has been added.
- This subsystem, located in drivers/pinctrl/ provides a generic subsystem to handle pin muxing. It offers:
  - A pin muxing driver interface, to implement the system-on-chip specific drivers that configure the muxing.
  - A pin muxing consumer interface, for device drivers.
- Most *pinctrl* drivers provide a Device Tree binding, and the pin muxing must be described in the Device Tree.
  - The exact Device Tree binding depends on each driver. Each binding is defined in Documentation/devicetree/bindings/pinctrl.



pinctrl subsystem diagram





Device Tree properties for consumer devices

The devices that require certains pins to be muxed will use the `pinctrl-<x>` and `pinctrl-names` Device Tree properties.

- The `pinctrl-0`, `pinctrl-1`, `pinctrl-<x>` properties link to a pin configuration for a given state of the device.
- The `pinctrl-names` property associates a name to each state. The name `default` is special, and is automatically selected by a device driver, without having to make an explicit *pinctrl* function call.
- See Documentation/devicetree/bindings/pinctrl/pinctrl-bindings.txt for details.



Device Tree properties for consumer devices - Examples

```
i2c0: i2c@11000 {
        ...
        pinctrl-0 = <&pmx_twsi0>;
        pinctrl-names = "default";
        ...
};
```

Most common case (arch/arm/boot/dts/marvell/kirkwood.dtsi)

```
i2c0: i2c@f8014000 {
        ...
        pinctrl-names = "default", "gpio";
        pinctrl-0 = <&pinctrl_i2c0>;
        pinctrl-1 = <&pinctrl_i2c0_gpio>;
        ...
};
```

Case with multiple pin states (arch/arm/boot/dts/microchip/sama5d4.dtsi)



Defining pinctrl configurations

- The different *pinctrl configurations* must be defined as child nodes of the main *pinctrl device* (which controls the muxing of pins).
- The configurations may be defined at:
  - the SoC level (`.dtsi` file), for pin configurations that are often shared between multiple boards
  - at the board level (`.dts` file) for configurations that are board specific.
- The `pinctrl-<x>` property of the consumer device points to the pin configuration it needs through a DT *phandle*.
- The description of the configurations is specific to each *pinctrl driver*. See Documentation/devicetree/bindings/pinctrl for the pinctrl bindings.



Example on OMAP/AM33xx

- On OMAP/AM33xx, the `pinctrl-single` driver is used. It is common between multiple SoCs and simply allows to configure pins by writing a value to a register.
  - In each pin configuration, a `pinctrl-single,pins` value gives a list of *(register, value)* pairs needed to configure the pins.
- To know the correct values, one must use the SoC and board datasheets.

```
/* Excerpt from am335x-bone-common.dts */

&am33xx_pinmux {
   ...
   i2c2_pins: pinmux_i2c2_pins {
      pinctrl-single,pins = <
         /* uart1_ctsn.i2c2_sda */
         AM33XX_PADCONF(AM335X_PIN_UART1_CTSN,
                        PIN_INPUT_PULLUP, MUX_MODE3)
         /* uart1_rtsn.i2c2_scl */
         AM33XX_PADCONF(AM335X_PIN_UART1_RTSN,
                        PIN_INPUT_PULLUP, MUX_MODE3)
      >;
   };
};

&i2c2 {
   pinctrl-names = "default";
   pinctrl-0 = <&i2c2_pins>;

   status = "okay";
   clock-frequency = <400000>;
   ...

   pressure@76 {
      compatible = "bosch,bmp280";
      reg = <0x76>;
   };
};
```



Example on the Allwinner A20 SoC

SoC level

```
/ {
    ...
    soc {
        ...
        pio: pinctrl@01c20800 {
            compatible = "allwinner,sun7i-a20-pinctrl";
            reg = <0x01c20800 0x400>;
            ...

            uart0_pb_pins: uart0-pb-pins {
                pins = "PB22", "PB23";
                function = "uart0";
            };
            ...
        };
    };
};
```

UART0 pin mux config

arch/arm/boot/dts/sun7i-a20.dtsi

Board level

```
/ {
...
    leds {
        compatible = "gpio-leds";
        pinctrl-names = "default";
        pinctrl-0 = <&led_pins_olinuxino>;

        led {
            label = "a20-olinuxino-micro:green:usr";
            gpios = <&pio 7 2 GPIO_ACTIVE_HIGH>;
            default-state = "on";
        };
    };
...
};


&pio {
    ....

    led_pins_olinuxino: led_pins@0 {
        pins = "PH2";
        function = "gpio_out";
        drive-strength = <20>;
    };

...
};

&uart0 {
    pinctrl-names = "default";
    pinctrl-0 = <&uart0_pb_pins>;
    status = "okay";
};
```

Declare LED device and associate pin mux config

LED pin mux config

Enable UART0 and associate pin mux config

arch/arm/boot/dts/sun7i-a20-olinuxino-micro.dts

Practical lab - Setup pinmuxing to enable I2C communication



- Configure the pinmuxing for the I2C bus used to communicate with the Nunchuk
- Validate that the I2C communication works with user space tools.

Linux device and driver model



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Introduction



The need for a device model?

- The Linux kernel runs on a wide range of architectures and hardware platforms, and therefore needs to **maximize the reusability** of code between platforms.
- For example, we want the same *USB device driver* to be usable on a x86 PC, or an ARM platform, even though the USB controllers used on these platforms are different.
- This requires a clean organization of the code, with the *device drivers* separated from the *controller drivers*, the hardware description separated from the drivers themselves, etc.
- This is what the Linux kernel **Device Model** allows, in addition to other advantages covered in this section.

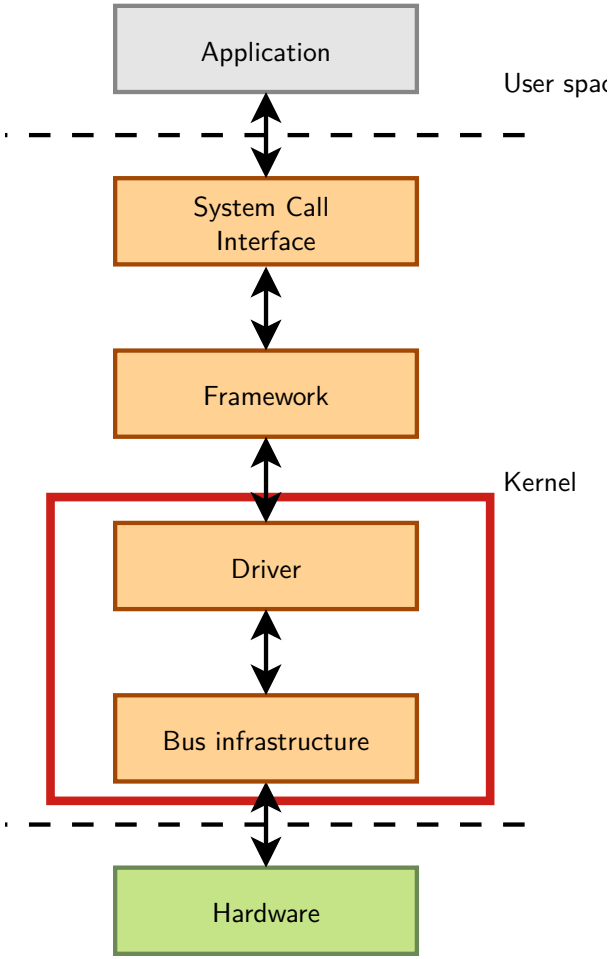


Kernel and device drivers

In Linux, a driver is always interfacing with:

- a **framework** that allows the driver to expose the hardware features in a generic way.
- a **bus infrastructure**, part of the device model, to detect/communicate with the hardware.

This section focuses on the *bus infrastructure*, while *kernel frameworks* are covered later in this training.





Device model data structures

- The *device model* is organized around three main data structures:
  - The struct bus_type structure, which represents one type of bus (USB, PCI, I2C, etc.)
  - The struct device_driver structure, which represents one driver capable of handling certain devices on a certain bus.
  - The struct device structure, which represents one device connected to a bus
- The kernel uses inheritance to create more specialized versions of struct device_driver and struct device for each bus subsystem.



Bus drivers

- The first component of the device model is the bus driver
  - One bus driver for each type of bus: USB, PCI, SPI, MMC, I2C, etc.
- It is responsible for
  - Registering the bus type (struct bus_type)
  - Allowing the registration of adapter drivers (USB controllers, I2C adapters, etc.), able to detect the connected devices (if possible), and providing a communication mechanism with the devices
  - Allowing the registration of device drivers (USB devices, I2C devices, PCI devices, etc.), managing the devices
  - Matching the device drivers against the devices detected by the adapter drivers.
  - Provides an API to implement both adapter drivers and device drivers
  - Defining driver and device specific structures, eg. struct usb_driver and struct usb_interface



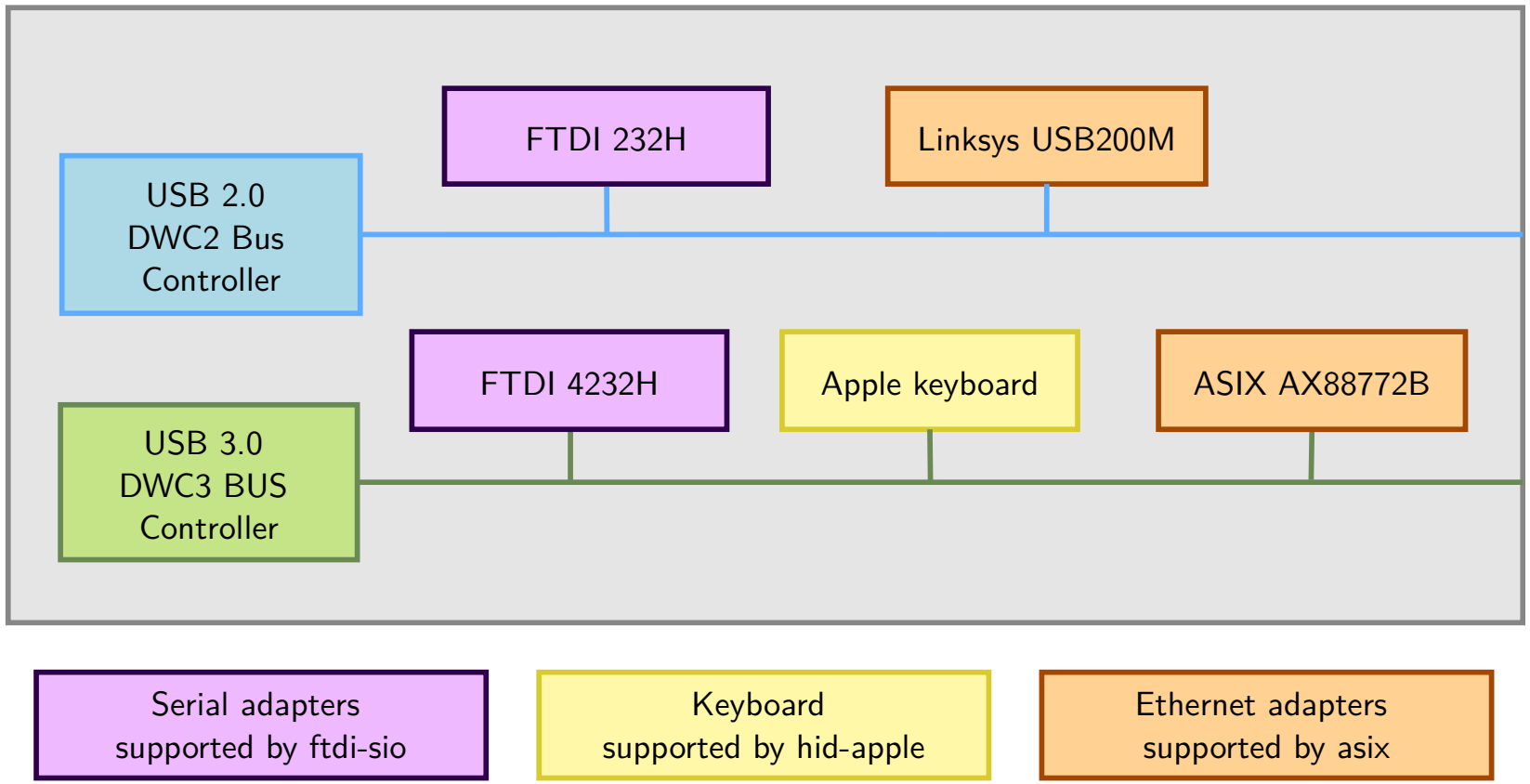
sysfs

- The bus, device, drivers, etc. structures are internal to the kernel
- The `sysfs` virtual filesystem offers a mechanism to export such information to user space
- Used for example by `udev` to provide automatic module loading, firmware loading, mounting of external media, etc.
- `sysfs` is usually mounted in `/sys`
  - `/sys/bus/` contains the list of buses
  - `/sys/devices/` contains the list of devices
  - `/sys/class` enumerates devices by the framework they are registered to (`net`, `input`, `block`...), whatever bus they are connected to. Very useful!



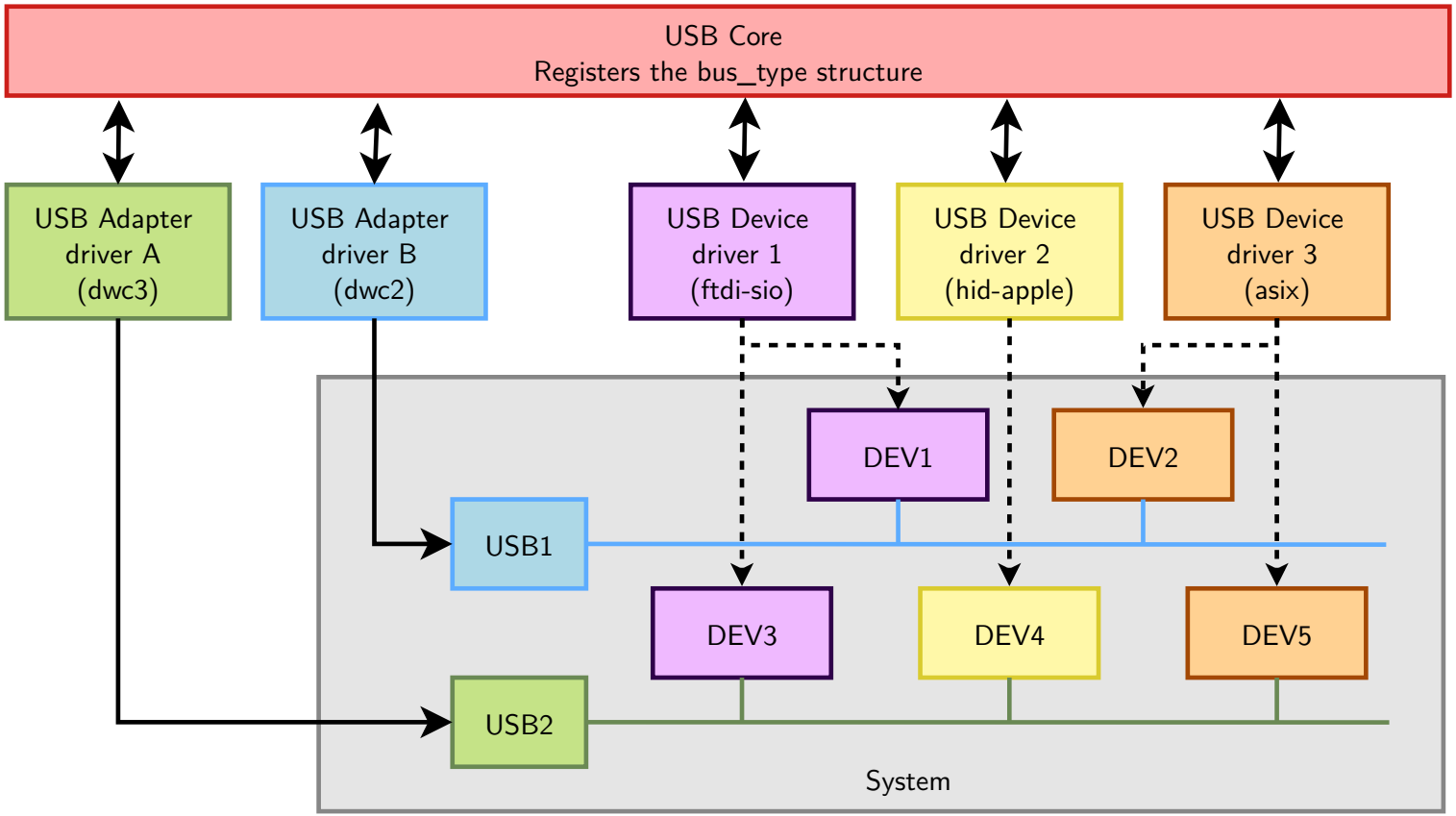
Example of the USB bus

Example: USB bus 1/3



Hardware view of the bus

Example: USB bus 2/3



Device model view of the bus



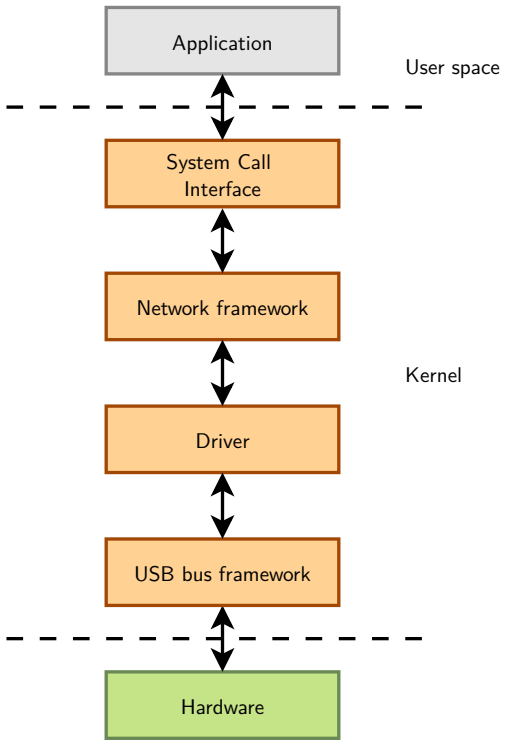
Example: USB bus 3/3

- Core infrastructure (bus driver)
  - drivers/usb/core/
  - struct bus_type is defined in drivers/usb/core/driver.c and registered in drivers/usb/core/usb.c
- Adapter drivers
  - drivers/usb/host/
  - For EHCI, UHCI, OHCI, XHCI, and their implementations on various systems (Microchip, IXP, Xilinx, OMAP, Samsung, PXA, etc.)
- Device drivers
  - Everywhere in the kernel tree, classified by their type (Example: drivers/net/usb/)



Example of device driver

- To illustrate how drivers are implemented to work with the device model, we will study the source code of a driver for a USB network card
  - It is USB device, so it has to be a USB device driver
  - It exposes a network device, so it has to be a network driver
  - Most drivers rely on a bus infrastructure (here, USB) and register themselves in a framework (here, network)
- We will only look at the device driver side, and not the adapter driver side
- The driver we will look at is drivers/net/usb/rtl8150.c





Device identifiers

- Defines the set of devices that this driver can manage, so that the USB core knows for which devices this driver should be used
- The MODULE_DEVICE_TABLE() macro allows `depmod` (run by `make modules_install`) to extract the relationship between, device identifiers and drivers, so that drivers can be loaded automatically by `udev`. See `/lib/modules/$(uname -r)/modules.{alias, usbmap}`

```
static struct usb_device_id rtl8150_table[] = {
    { USB_DEVICE(VENDOR_ID_REALTEK, PRODUCT_ID_RTL8150) },
    { USB_DEVICE(VENDOR_ID_MELCO, PRODUCT_ID_LUAKTX) },
    { USB_DEVICE(VENDOR_ID_MICRONET, PRODUCT_ID_SP128AR) },
    { USB_DEVICE(VENDOR_ID_LONGSHINE, PRODUCT_ID_LCS8138TX) },
    [...]
    {}
}; MODULE_DEVICE_TABLE(usb, rtl8150_table);
```



Instantiation of usb_driver

- struct usb_driver is a structure defined by the USB core. Each USB device driver must instantiate it, and register itself to the USB core using this structure
- This structure inherits from struct device_driver, which is defined by the device model.

```
static struct usb_driver rtl8150_driver = {
    .name = "rtl8150",
    .probe = rtl8150_probe,
    .disconnect = rtl8150_disconnect,
    .id_table = rtl8150_table,
    .suspend = rtl8150_suspend,
    .resume = rtl8150_resume
};
```



Driver registration and unregistration

- When the driver is loaded / unloaded, it must register / unregister itself to / from the USB core
- Done using usb_register() and usb_deregister(), provided by the USB core.

```
static int __init usb_rtl8150_init(void)
{
    return usb_register(&rtl8150_driver);
}

static void __exit usb_rtl8150_exit(void)
{
    usb_deregister(&rtl8150_driver);
}

module_init(usb_rtl8150_init);
module_exit(usb_rtl8150_exit);
```

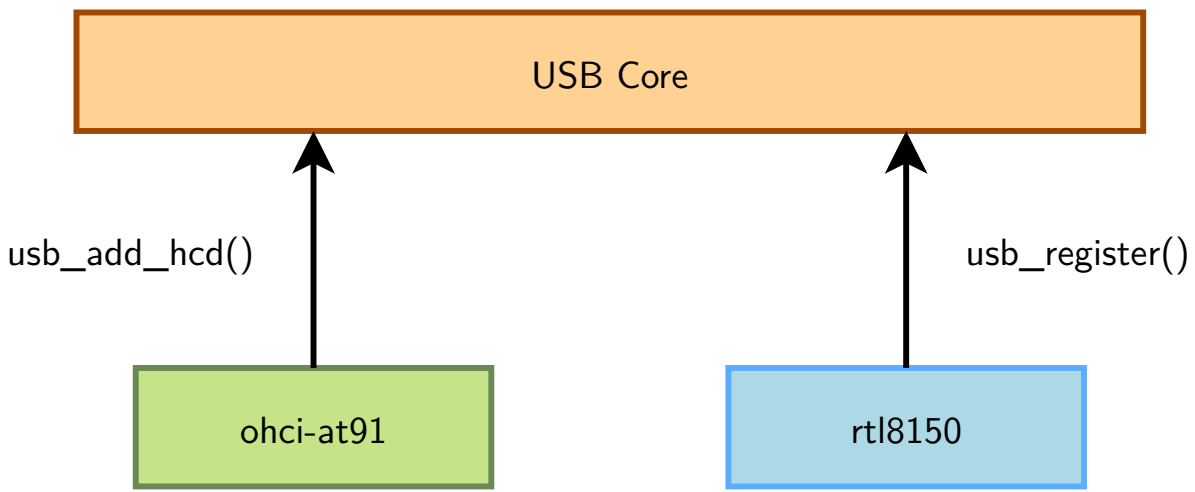
- All this code is actually replaced by a call to the module_usb_driver() macro:

```
module_usb_driver(rtl8150_driver);
```



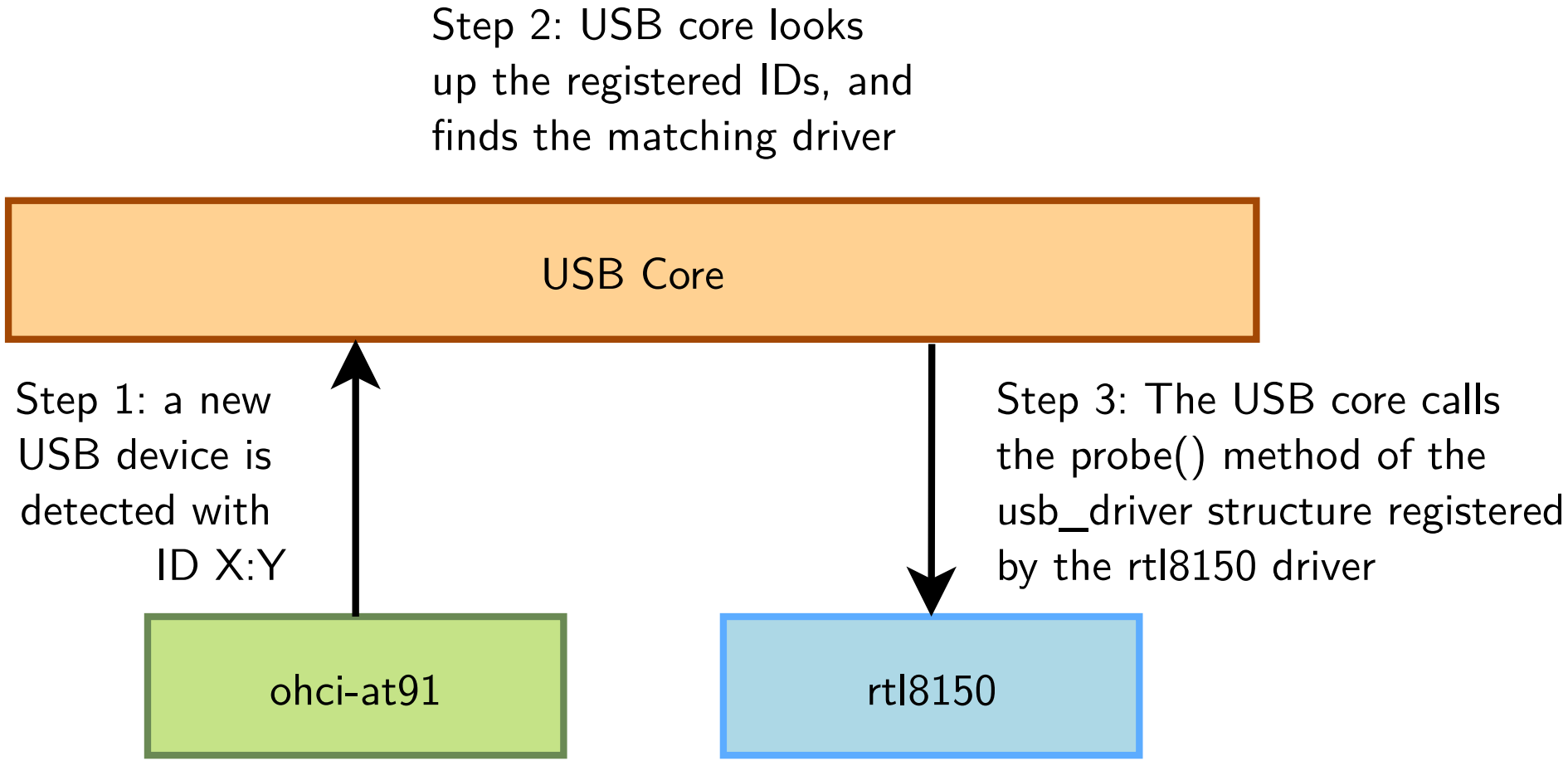
At Initialization

- The USB adapter driver that corresponds to the USB controller of the system registers itself to the USB core
- The rtl8150 USB device driver registers itself to the USB core



- The USB core now knows the association between the vendor/product IDs of rtl8150 and the struct usb_driver structure of this driver

When a device is detected





Probe method

- Invoked **for each device** bound to a driver
- The `probe()` method receives as argument a structure describing the device, usually specialized by the bus infrastructure (struct pci_dev, struct usb_interface, etc.)
- This function is responsible for
  - Initializing the device, mapping I/O memory, registering the interrupt handlers. The bus infrastructure provides methods to get the addresses, interrupt numbers and other device-specific information.
  - Registering the device to the proper kernel framework, for example the network infrastructure.



Example: probe() and disconnect() methods

```
static int
rtl8150_probe(struct usb_interface *intf,
              const struct usb_device_id *id)
{
    rtl8150_t *dev;
    struct net_device *netdev;

    netdev = alloc_etherdev(sizeof(rtl8150_t));
    [...]
    dev = netdev_priv(netdev);
    tasklet_init(&dev->tl, rx_fixup,
                 (unsigned long)dev);
    spin_lock_init(&dev->rx_pool_lock);
    [...]
    netdev->netdev_ops = &rtl8150_netdev_ops;
    alloc_all_urbs(dev);
    [...]
    usb_set_intfdata(intf, dev);
    SET_NETDEV_DEV(netdev, &intf->dev);
    register_netdev(netdev);

    return 0;
}
```

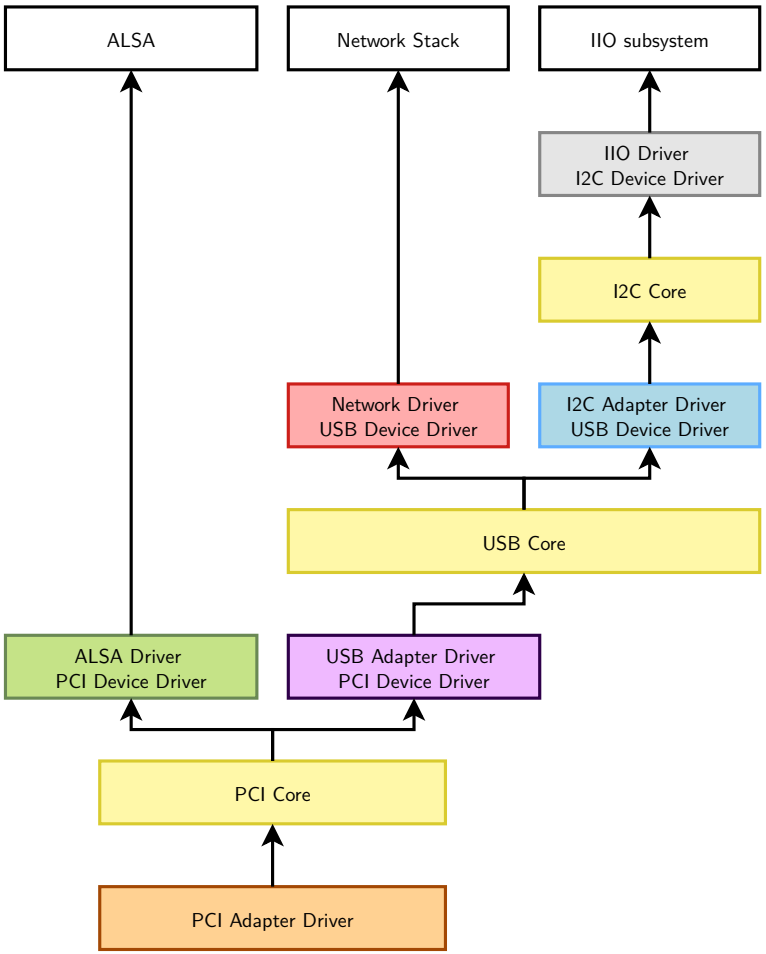
```
static void
rtl8150_disconnect(struct usb_interface *intf)
{
        rtl8150_t *dev = usb_get_intfdata(intf);

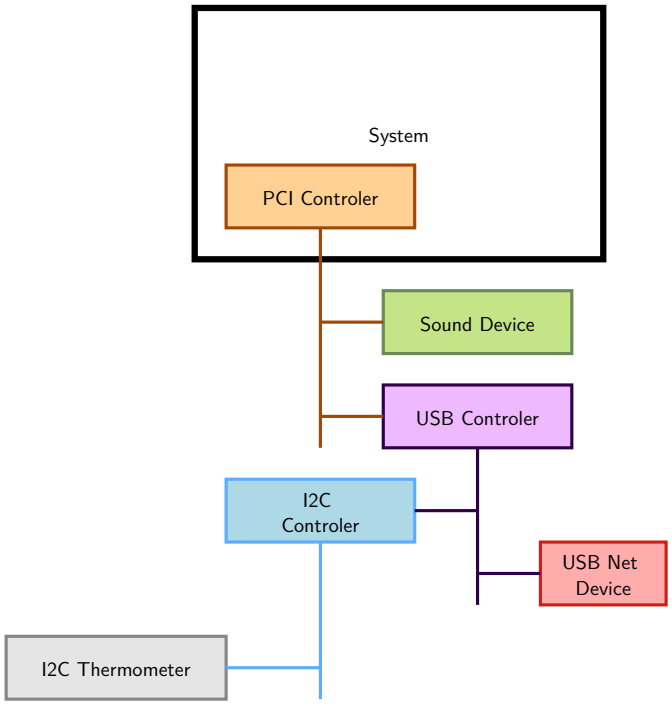
        usb_set_intfdata(intf, NULL);
        if (dev) {
                set_bit(RTL8150_UNPLUG, &dev->flags);
                tasklet_kill(&dev->tl);
                unregister_netdev(dev->netdev);
                unlink_all_urbs(dev);
                free_all_urbs(dev);
                free_skb_pool(dev);
                if (dev->rx_skb)
                        dev_kfree_skb(dev->rx_skb);
                kfree(dev->intr_buff);
                free_netdev(dev->netdev);
        }
}
```

Source: drivers/net/usb/rtl8150.c



The model is recursive







Platform drivers



Platform devices

- Amongst the non-discoverable devices, a huge family are the devices that are directly part of a system-on-chip: UART controllers, Ethernet controllers, SPI or I2C controllers, graphic or audio devices, etc.
- In the Linux kernel, a special bus, called the **platform bus** has been created to handle such devices. Those get controlled through **memory-mapped registers**.
- It supports **platform drivers** that handle **platform devices**.
- It works like any other bus (USB, PCI), except that devices are enumerated statically instead of being discovered dynamically.



Implementation of a platform driver (1)

The driver implements a struct platform_driver structure (example taken from drivers/tty/serial/imx.c, simplified)

```
static struct platform_driver serial_imx_driver = {
        .probe          = serial_imx_probe,
        .remove         = serial_imx_remove,
        .id_table       = imx_uart_devtype,
        .driver         = {
                .name   = "imx-uart",
                .of_match_table = imx_uart_dt_ids,
                .pm     = &imx_serial_port_pm_ops,
        },
};
```



Implementation of a platform driver (2)

... and registers its driver to the platform driver infrastructure

```
static int __init imx_serial_init(void) {
    return platform_driver_register(&serial_imx_driver);
}

static void __exit imx_serial_cleanup(void) {
    platform_driver_unregister(&serial_imx_driver);
}

module_init(imx_serial_init);
module_exit(imx_serial_cleanup);
```

Most drivers actually use the module_platform_driver() macro when they do nothing special in `init()` and `exit()` functions:

```
module_platform_driver(serial_imx_driver);
```



Platform device instantiation

- As platform devices cannot be detected dynamically, they are defined statically
  - Legacy way: by direct instantiation of struct platform_device structures, as done on a few old ARM platforms. The device was part of a list, and the list of devices was added to the system during board initialization.
  - Current way: by parsing an "external" description, like a *device tree* on most embedded platforms today, from which struct platform_device instances are created.



Using additional hardware resources

- Regular DT descriptions contain many information. It includes phandles (pointers) towards additional hardware blocks which cannot be discovered.
  - I/O register addresses and IRQ lines are available through a struct resource array associated to each struct platform_device.
  - Information relevant to a given subsystem is parsed by that specific subsystem. Examples are clocks, GPIOs or DMA. A subsystem is responsible for:
    - instantiating its components,
    - offering an API to use those objects from device drivers.
  - Specific properties are directly retrieved by device drivers, through (expensive) DT lookups.
- All these methods allow the same driver to be used with multiple devices functioning similarly, but with different addresses, IRQs, etc.



Using resources

- The platform driver has access to the resources provided by the platform bus:

```
res = platform_get_resource(pdev, IORESOURCE_MEM, 0);
sport->rxirq = platform_get_irq(pdev, 0);
```

- As well as the various subsystem-provided dependencies through individual APIs:
  - clk_get()
  - gpio_request()
  - dma_request_channel()



Driver data

- In addition to the per-device resources and information, drivers may require driver-specific information to behave slightly differently when different flavors of an IP block are driven by the same driver.
- A `const void *data` pointer can be used to store per-compatible specificities:

```
static const struct of_device_id marvell_nfc_of_ids[] = {
        {
                .compatible = "marvell,armada-8k-nand-controller",
                .data = &marvell_armada_8k_nfc_caps,
        },
};
```

- Which can be retrieved in the probe with:

```
/* Get NAND controller capabilities */
if (pdev->id_entry) /* legacy way */
        nfc->caps = (void *)pdev->id_entry->driver_data;
else /* current way */
        nfc->caps = of_device_get_match_data(&pdev->dev);
```